In [2]:
```
import pandas as pd
import numpy as np
```

In [3]:
```
df = pd.read_csv("emails.csv")
```

In [4]:
```
df.head()
```

Out[4]:

| | Email No. | the | to | ect | and | for | of | a | you | hou | ... | connevey | jay | valued | lay | i |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | Email 1 | 0 | 0 | 1 | 0 | 0 | 0 | 2 | 0 | 0 | ... | 0 | 0 | 0 | 0 | |
| **1** | Email 2 | 8 | 13 | 24 | 6 | 6 | 2 | 102 | 1 | 27 | ... | 0 | 0 | 0 | 0 | |
| **2** | Email 3 | 0 | 0 | 1 | 0 | 0 | 0 | 8 | 0 | 0 | ... | 0 | 0 | 0 | 0 | |
| **3** | Email 4 | 0 | 5 | 22 | 0 | 5 | 1 | 51 | 2 | 10 | ... | 0 | 0 | 0 | 0 | |
| **4** | Email 5 | 7 | 6 | 17 | 1 | 5 | 2 | 57 | 0 | 9 | ... | 0 | 0 | 0 | 0 | |

5 rows × 3002 columns

In [8]:
```
df.info()
df.isnull().sum().sum()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5172 entries, 0 to 5171
Columns: 3002 entries, Email No. to Prediction
dtypes: int64(3001), str(1)
memory usage: 118.5 MB
```

Out[8]: np.int64(0)

In [7]:
```
df.describe()
```

Out[7]:

| | the | to | ect | and | for | of | |
|---|---|---|---|---|---|---|---|
| **count** | 5172.000000 | 5172.000000 | 5172.000000 | 5172.000000 | 5172.000000 | 5172.000000 | 5 |
| **mean** | 6.640565 | 6.188128 | 5.143852 | 3.075599 | 3.124710 | 2.627030 | |
| **std** | 11.745009 | 9.534576 | 14.101142 | 6.045970 | 4.680522 | 6.229845 | |
| **min** | 0.000000 | 0.000000 | 1.000000 | 0.000000 | 0.000000 | 0.000000 | |
| **25%** | 0.000000 | 1.000000 | 1.000000 | 0.000000 | 1.000000 | 0.000000 | |
| **50%** | 3.000000 | 3.000000 | 1.000000 | 1.000000 | 2.000000 | 1.000000 | |
| **75%** | 8.000000 | 7.000000 | 4.000000 | 3.000000 | 4.000000 | 2.000000 | |
| **max** | 210.000000 | 132.000000 | 344.000000 | 89.000000 | 47.000000 | 77.000000 | 1 |

8 rows × 3001 columns

In [10]:
```
X = df.iloc[:, 1:-1].values
y = df.iloc[:, -1].values
```

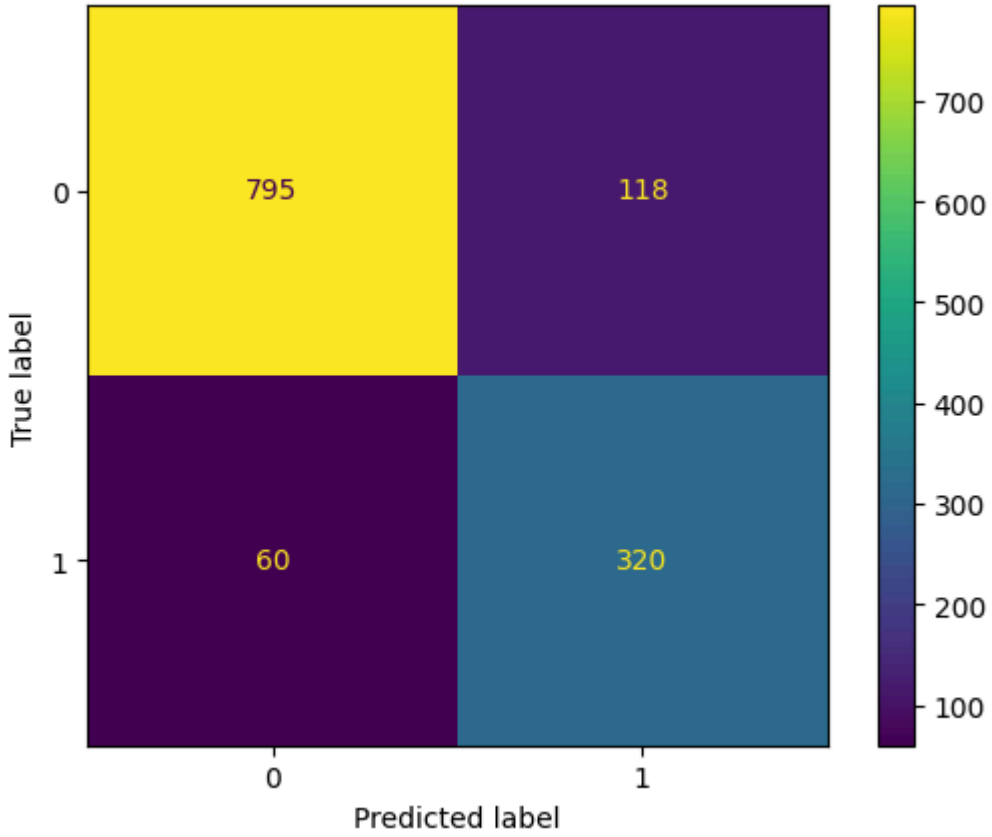
In [15]:
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix, accuracy_s
x_train, x_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)
```

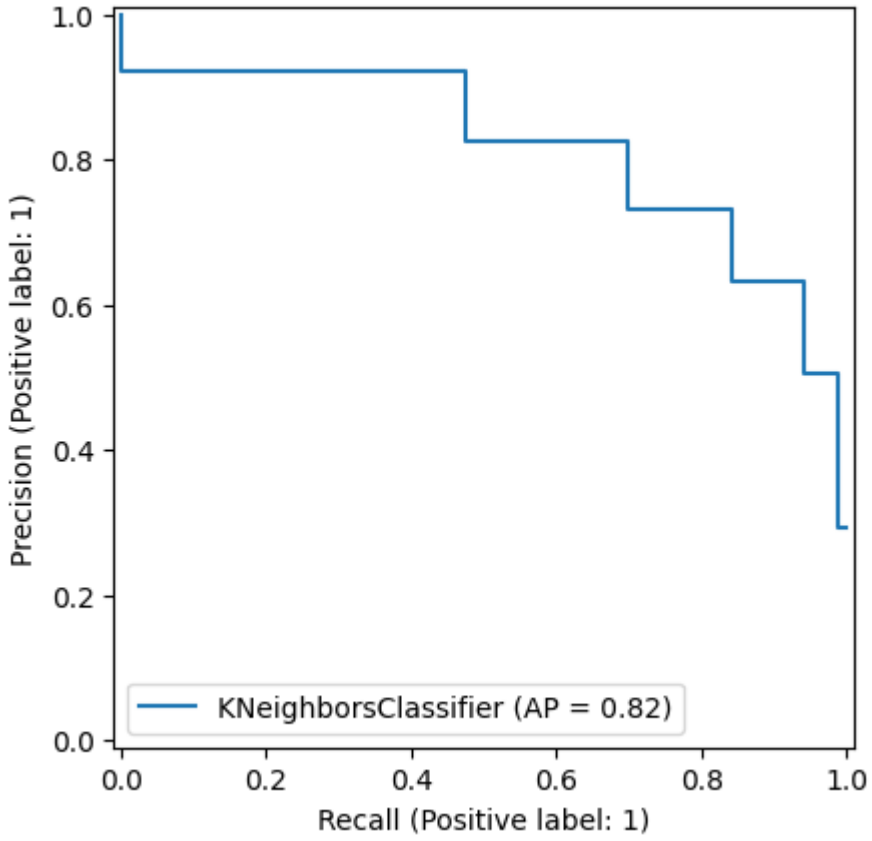
In [16]:
```
def report(classifier):
    y_pred = classifier.predict(x_test)
    cm = confusion_matrix(y_test,y_pred)
    display = ConfusionMatrixDisplay(cm,display_labels = classifier.classes_)
    display.plot()
    print(f"Accuracy : {accuracy_score(y_test,y_pred)}")
    print(f"Precision Score :{precision_score(y_test,y_pred)}")
    print(f"Recall score : {recall_score(y_test,y_pred)}")
    PrecisionRecallDisplay.from_estimator(classifier,x_test,y_test)
    RocCurveDisplay.from_estimator(classifier,x_test,y_test)
```

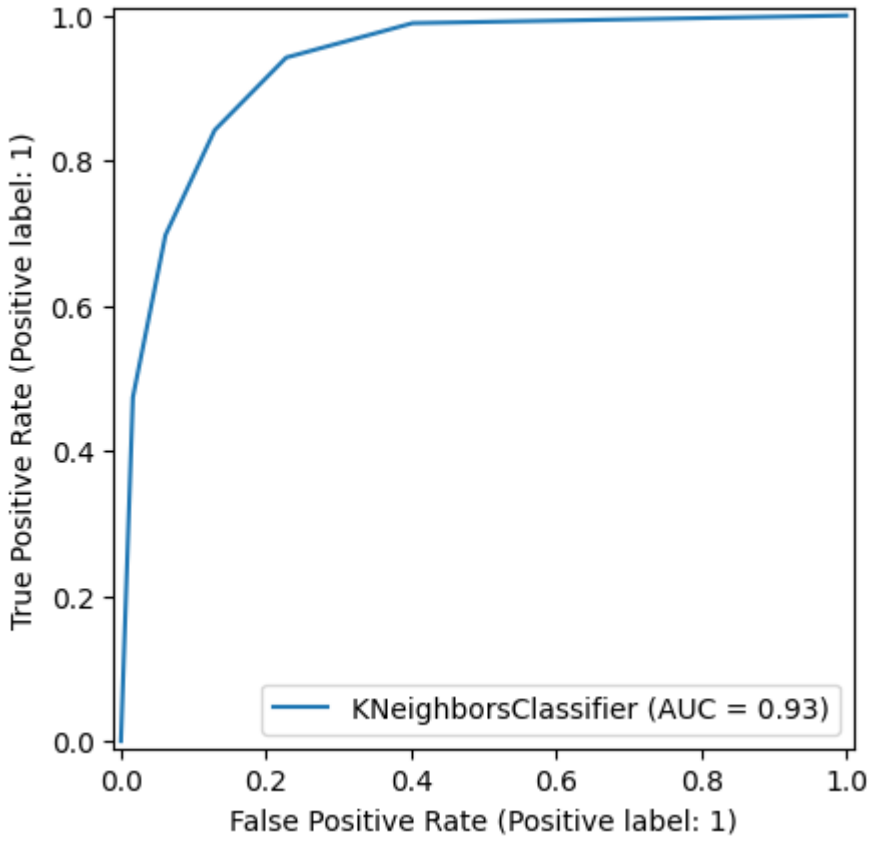
In [17]:
```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(x_train, y_train)
report(knn)
```

```
Accuracy : 0.8623356535189481
Precision Score :0.730593607305936
Recall score : 0.8421052631578947
```


795
118
60
320
0
1
True label
0
1
Predicted label
700
600
500
400
300
200
100

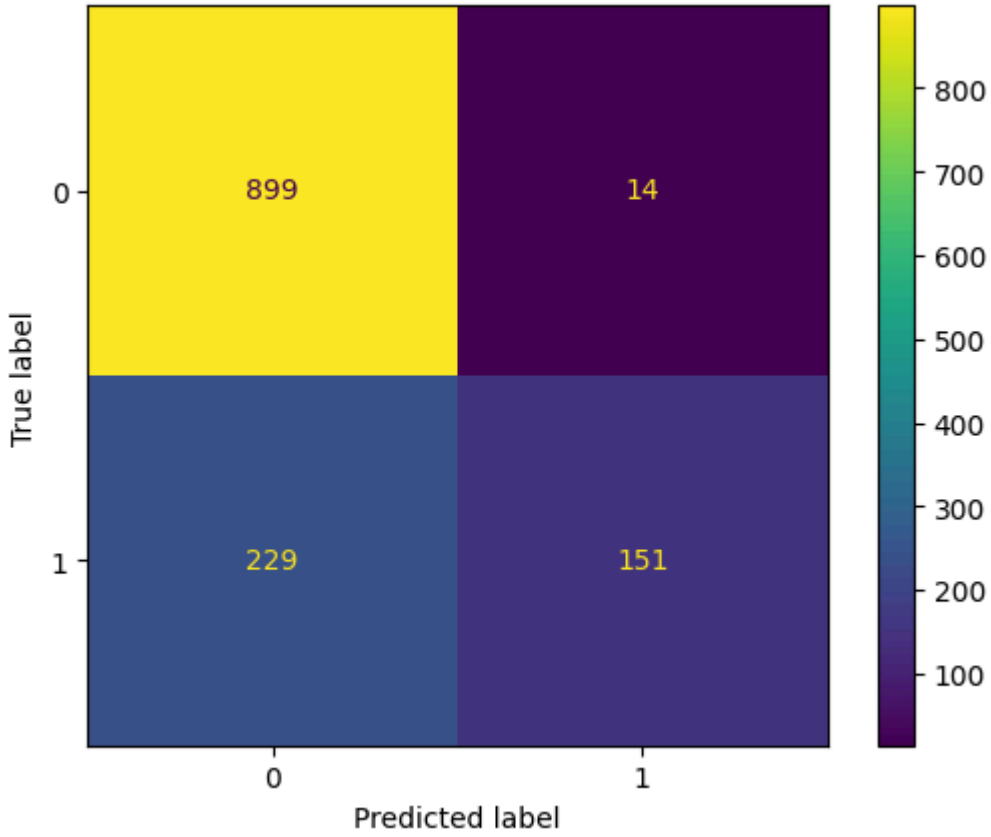

Precision (Positive label: 1)
Recall (Positive label: 1)
KNeighborsClassifier (AP = 0.82)
0.0
0.2
0.4
0.6
0.8
1.0

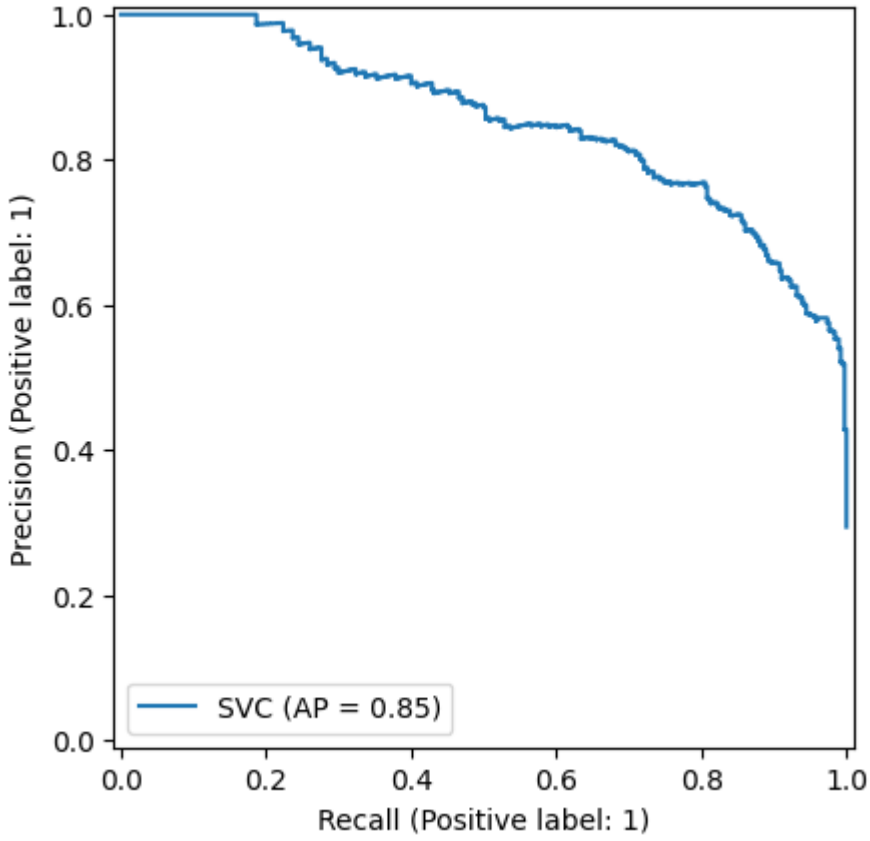


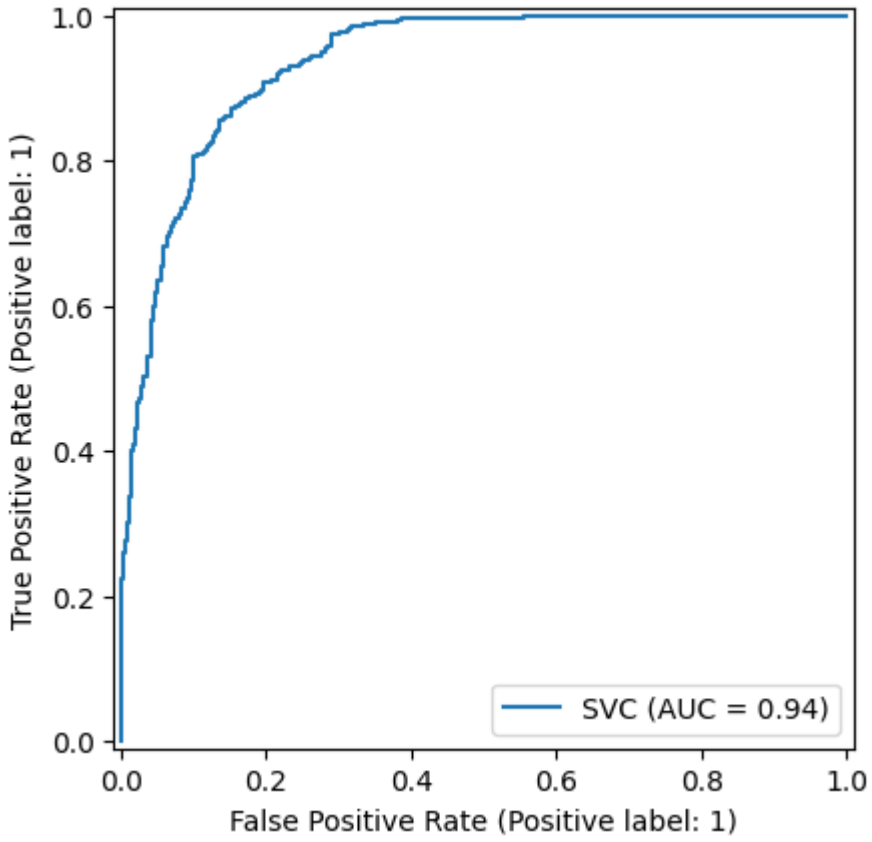

In [18]:
```
from sklearn.svm import SVC

svm = SVC(kernel='rbf', random_state=42)
svm.fit(x_train, y_train)
report(svm)
```

```
Accuracy : 0.8120649651972158
Precision Score :0.9151515151515152
Recall score : 0.3973684210526316
```


899
14
229
151
0
1
True label
0
1
Predicted label
800
700
600
500
400
300
200
100


1.0
0.8
0.6
0.4
0.2
0.0
Precision (Positive label: 1)
SVC (AP = 0.85)
0.0
0.2
0.4
0.6
0.8
1.0
Recall (Positive label: 1)


1.0
0.8
0.6
0.4
0.2
0.0
True Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
False Positive Rate (Positive label: 1)
SVC (AUC = 0.94)

In [ ]: